



Metodología de Sistemas I

Del Relevamiento de Requerimientos a la Agilidad con Scrum

Tecnicatura Universitaria en Programación - UTN

Agenda del Encuentro

Un recorrido completo por las metodologías fundamentales del desarrollo de software moderno, desde la captura inicial de necesidades hasta la implementación de marcos de trabajo ágiles.

01

Ingeniería de Requerimientos

Relevamiento y clasificación de necesidades del sistema

03

Historias de Usuario

Captura ágil de requerimientos desde la perspectiva del usuario

05

Scrum en Profundidad

Roles, eventos y artefactos del marco de trabajo

02

Modelado del Sistema

Casos de Uso y diagramas UML para visualizar la solución

04

Introducción a la Agilidad

Manifiesto Ágil y pensamiento Lean

06

Gestión Visual

Implementación de tableros Kanban

¿Qué es el Relevamiento de Requerimientos?

El relevamiento de requerimientos es el proceso fundamental que consiste en identificar, documentar y comprender las necesidades reales del cliente o usuario final. Es la base sobre la cual se construye todo el sistema de software.

Técnicas Principales de Relevamiento

- **Entrevistas y cuestionarios:** Conversaciones estructuradas con stakeholders para capturar necesidades específicas
- **Observación directa:** Análisis del trabajo real de los usuarios en su entorno natural
- **Análisis de documentación:** Revisión de procesos, manuales y sistemas existentes

Un relevamiento efectivo requiere habilidades técnicas y de comunicación, estableciendo las bases para el éxito del proyecto.



Clasificación de Requerimientos

Los requerimientos se clasifican en dos categorías fundamentales que definen distintos aspectos del sistema a desarrollar.

Requerimientos Funcionales

Describen **qué** debe hacer el sistema. Definen las funcionalidades, servicios y comportamientos específicos que el software debe proporcionar.

Ejemplo: "El sistema debe permitir registrar nuevos usuarios con email y contraseña"

Ejemplo: "El sistema debe generar reportes mensuales de ventas en formato PDF"

Requerimientos No Funcionales

Establecen restricciones de **cómo** debe funcionar el sistema. Definen atributos de calidad como rendimiento, seguridad, usabilidad y confiabilidad.

Ejemplo: "El sistema debe responder a consultas en menos de 2 segundos"

Ejemplo: "La aplicación debe soportar 1000 usuarios concurrentes"

Ambos tipos de requerimientos son esenciales para definir completamente el sistema y deben especificarse con precisión.

Calidad en la Especificación

Una especificación de requerimientos correcta y bien documentada es crucial para evitar errores costosos en las etapas posteriores del desarrollo. Los defectos en los requerimientos se amplifican exponencialmente durante la implementación.

Clara y Sin Ambigüedades

Cada requerimiento debe tener una única interpretación posible. Evitar términos vagos como "rápido", "eficiente" o "usuario amigable" sin definirlos cuantitativamente.

Comprobable y Verificable

Debe ser posible diseñar pruebas específicas que demuestren si el requerimiento fue implementado correctamente. Si no puede probarse, no es un buen requerimiento.

Consistente y Completa

No debe haber contradicciones entre requerimientos ni omisiones importantes. Todos los aspectos necesarios deben estar cubiertos sin información faltante.

Invertir tiempo en la calidad de los requerimientos reduce significativamente el retrabajo y aumenta las probabilidades de éxito del proyecto.

Introducción a los Casos de Uso



Los casos de uso constituyen una técnica central del análisis orientado a objetos. Representan las interacciones entre usuarios (actores) y el sistema para lograr un objetivo específico y con valor.

Elementos Fundamentales

- **Actor:** Quien interactúa con el sistema, puede ser un usuario humano o un sistema externo. Representa un rol, no una persona específica.
- **Caso de Uso:** Una secuencia de acciones que el sistema realiza para producir un resultado observable de valor para el actor.
- **Escenario:** Los pasos específicos que describen cómo se logra el objetivo, desde el inicio hasta la conclusión exitosa.

Esta técnica permite capturar los requerimientos funcionales desde la perspectiva del usuario, manteniendo el foco en el valor entregado.

Estructura de un Caso de Uso

Un caso de uso bien documentado contempla tanto el escenario ideal como las situaciones excepcionales que pueden ocurrir.



Flujo Básico (Escenario Principal)

La secuencia ideal de pasos, sin errores ni excepciones. Describe el camino más común y exitoso desde el inicio hasta alcanzar el objetivo del caso de uso.



Flujos Alternativos

Desvíos del flujo principal que contemplan errores, validaciones fallidas o caminos opcionales. Ejemplo: "El libro solicitado no está disponible" o "Credenciales incorrectas".

- ❏ **Ejemplo Práctico:** Actor "Estudiante" inicia sesión en el sistema bibliotecario, busca un libro específico por título o autor, visualiza la disponibilidad y realiza una reserva. Si el libro no está disponible, el sistema ofrece alternativas similares o permite agregar el título a una lista de espera.

Modelado Visual con UML

Unified Modeling Language

UML es el estándar internacional para representar gráficamente los diferentes aspectos de un sistema de software. Proporciona un lenguaje visual común que facilita la comunicación entre desarrolladores, analistas, diseñadores y stakeholders.

Permite visualizar la estructura, el comportamiento y las interacciones del sistema antes de escribir código, reduciendo ambigüedades y errores de interpretación.

Beneficios clave:

- Comunicación clara entre equipos técnicos y de negocio
- Documentación visual comprensible
- Detección temprana de inconsistencias
- Base para generación automática de código



Tipos de Diagramas UML

UML ofrece diversos tipos de diagramas, cada uno enfocado en representar diferentes aspectos del sistema.



Diagrama de Casos de Uso

Representa la funcionalidad del sistema desde la perspectiva externa del usuario. Muestra actores, casos de uso y sus relaciones, capturando los requerimientos funcionales de forma visual.



Diagrama de Clases

Describe la estructura estática del sistema: clases, sus atributos, métodos y las relaciones entre ellas (herencia, composición, asociación). Es fundamental para el diseño orientado a objetos.



Diagrama de Actividades

Modela los flujos de trabajo, la lógica de negocio y los procesos. Representa secuencias de acciones, decisiones y flujos paralelos, similar a un diagrama de flujo avanzado.



Historias de Usuario

Enfoque Ágil para Capturar Requerimientos

Las historias de usuario son una técnica ágil que captura requerimientos de forma breve y centrada en el valor para el usuario final. A diferencia de las especificaciones extensas, las historias son conversacionales y evolucionan mediante el diálogo continuo.

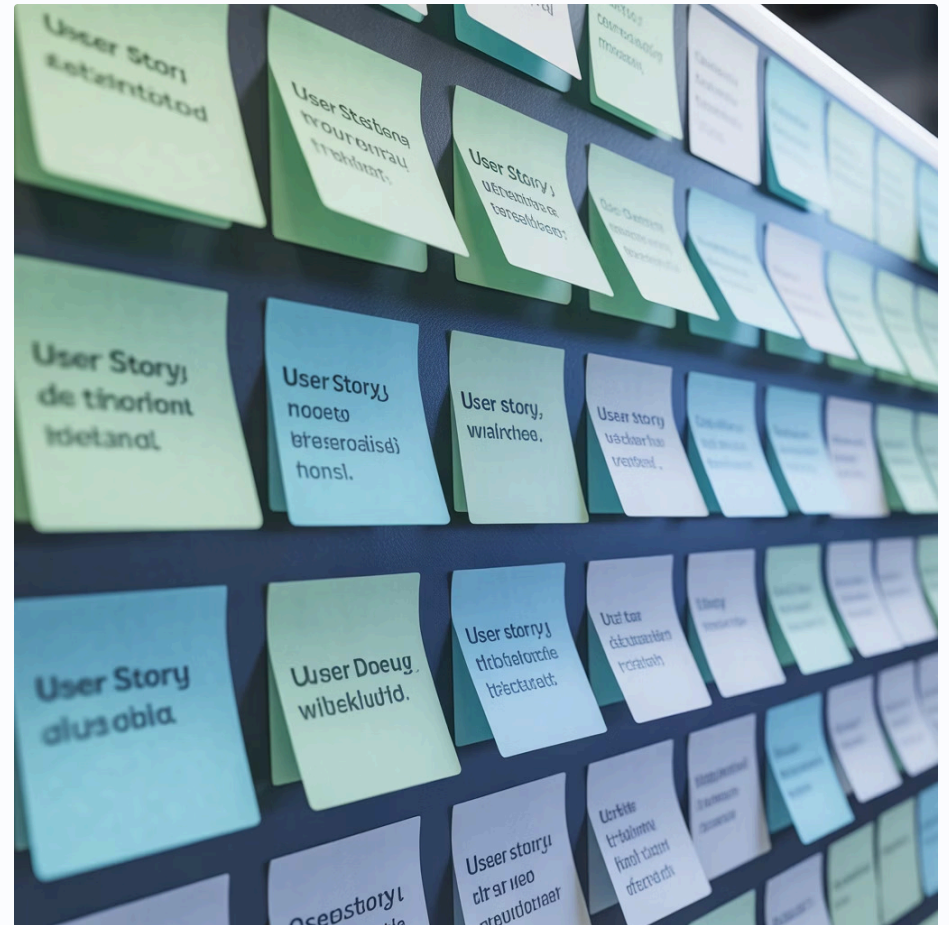
Formato Estándar

"**Como** [tipo de usuario], **quiero** [funcionalidad], **para** [beneficio o propósito]"

Ejemplos:

- "Como estudiante, quiero buscar libros por autor, para encontrar rápidamente las obras que necesito"
- "Como administrador, quiero generar reportes de préstamos, para analizar el uso de la biblioteca"

El objetivo es mantener el foco en el valor entregado y el propósito real detrás de cada funcionalidad, no en los detalles técnicos de implementación.



Criterios de Aceptación

Los criterios de aceptación acompañan a cada historia de usuario y son fundamentales para definir cuándo una funcionalidad se considera completa y correcta.

1

Definen el "Terminado"

Establecen las condiciones específicas que deben cumplirse para considerar la historia implementada exitosamente. Sin ambigüedades sobre qué significa "completado".

2

Guían las Pruebas

Sirven como base directa para crear casos de prueba. Cada criterio debe ser verificable mediante pruebas específicas, manuales o automatizadas.

3

Facilitan el Entendimiento Común

Aseguran que todo el equipo (desarrolladores, testers, Product Owner) comparten la misma visión sobre el resultado esperado.

❏ **Ejemplo para reserva de libro:** "El usuario debe estar autenticado en el sistema" • "El libro debe estar marcado como disponible" • "El sistema debe confirmar la reserva con un mensaje en pantalla" • "El usuario debe recibir un email de confirmación"

El Cambio de Paradigma: ¿Por qué Ágil?

El Problema con los Modelos Tradicionales

Los modelos tradicionales como cascada (waterfall) funcionaban en forma secuencial y rígida: primero se completaban todos los requerimientos, luego todo el diseño, después toda la implementación, y finalmente las pruebas.

Limitaciones principales:

- Poca flexibilidad ante cambios en los requerimientos
- Baja interacción con el cliente durante el desarrollo
- Entrega única al final del proyecto (alto riesgo)
- Detección tardía de problemas y errores
- Dificultad para adaptarse a mercados cambiantes

La Solución Ágil



Principios fundamentales:

- **Adaptabilidad:** Respuesta rápida a cambios inevitables
- **Entrega temprana de valor:** Funcionalidad usable en semanas, no meses
- **Feedback constante:** Aprendizaje continuo del cliente
- **Iteraciones cortas:** Reducción de riesgos mediante ciclos breves

El Manifiesto Ágil (2001)

En febrero de 2001, diecisiete desarrolladores se reunieron en Utah y crearon el Manifiesto Ágil, estableciendo los valores fundamentales que guían el desarrollo ágil de software.



Individuos e Interacciones

Sobre procesos y herramientas

Los equipos motivados y la comunicación efectiva son más valiosos que seguir procesos rígidos o depender de herramientas específicas.



Software Funcional

Sobre documentación extensiva

Un producto que funciona y aporta valor es mejor indicador de progreso que documentos voluminosos que describen lo que podría ser.



Colaboración con el Cliente

Sobre negociación contractual

Trabajar junto al cliente de forma continua genera mejores resultados que negociar contratos detallados al inicio y luego desconectarse.



Respuesta al Cambio

Sobre seguimiento de un plan

La capacidad de adaptarse a nuevas necesidades y aprendizajes es más valiosa que adherirse rígidamente a un plan inicial.

Estos valores no niegan la importancia de los elementos de la derecha, pero priorizan los de la izquierda cuando existe conflicto.

Pensamiento Lean



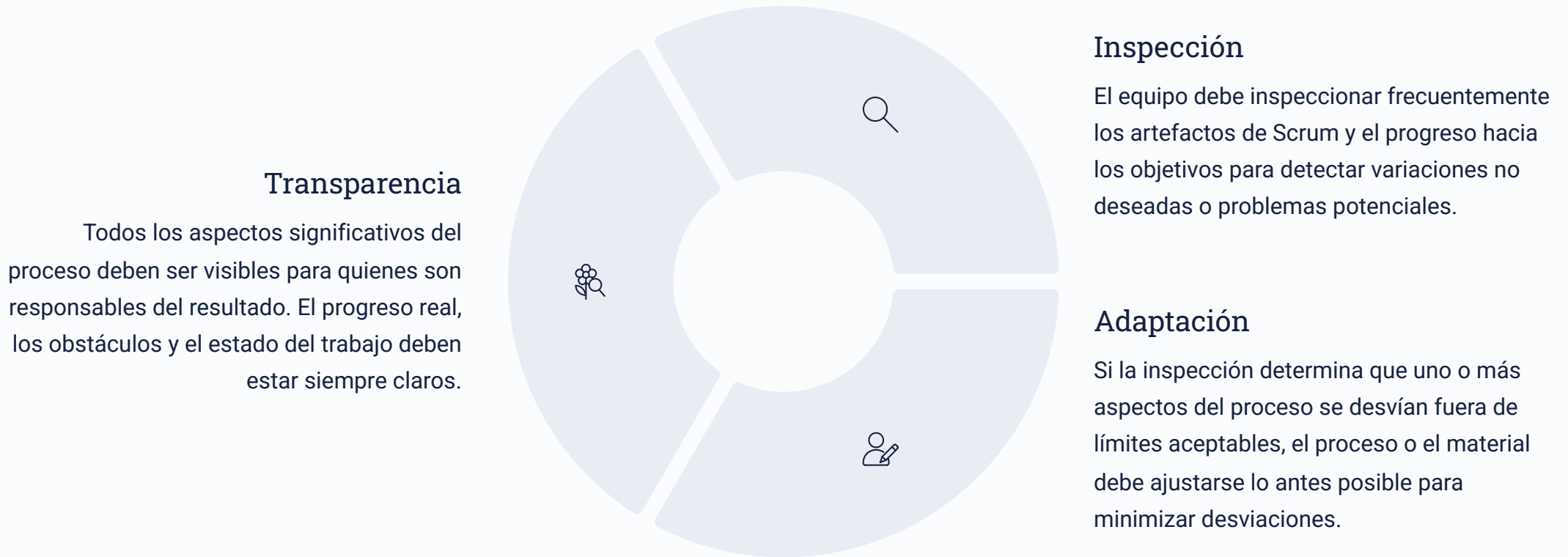
El pensamiento Lean tiene sus orígenes en la industria manufacturera japonesa, especialmente en el Sistema de Producción Toyota. Sus principios han sido adoptados por las metodologías ágiles de software.

Objetivos Integrados en Ágil

- **Eliminar desperdicios (Muda):** Cualquier actividad que no agrega valor al cliente debe minimizarse o eliminarse. Incluye trabajo innecesario, esperas, código no utilizado y reuniones improductivas.
- **Automatizar tareas repetitivas:** Liberar tiempo del equipo mediante automatización de pruebas, despliegues y procesos rutinarios.
- **Maximizar el valor entregado:** Enfocarse constantemente en lo que más beneficia al usuario final y al negocio.
- **Mejora continua (Kaizen):** Buscar incrementalmente formas de mejorar procesos y productos.

Introducción a Scrum

Scrum es el marco de trabajo ágil más popular, diseñado para gestionar proyectos complejos mediante iteraciones cortas y feedback continuo.



Se trabaja en ciclos breves llamados **Sprints**, típicamente de 2 a 4 semanas, al final de los cuales se entrega un incremento funcional del producto.